2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)

Dynamic Alert Suppression Policy for Noise Reduction in AIOps

Karan Bhukar
Harshit Kumar
Seema Nagar
Pooja Aggarwal
karan.bhukar1@ibm.com
harshitk@in.ibm.com
senagar3@in.ibm.com
aggarwal.pooja@in.ibm.com
IBM India Research Laboratory

Rohan Arora
Ruchi Mahindru
Amit Paradkar
rohan.arora@ibm.com
rmahindr@us.ibm.com
paradkar@us.ibm.com
IBM T.J. Watson Research Center, NY USA

ABSTRACT

As IT environments evolve in both size and complexity, observability tools are needed to monitor their health. As the anomalous events are detected, alerts are generated, leading to alert notifications to the Site Reliability Engineers(SREs). However, most of these notifications turn out to be false alarms, leading to alert fatigue, and inefficiencies. Existing approaches for reducing alert noise rely on static policies that can quickly become outdated in dynamic IT environments and are therefore difficult to maintain. In this work, we propose a novel unsupervised approach, Dynamic-X-Y, guided by a well known moving average envelope statistical method, to learn custom tailored alert suppression policy from historical alerts and events data. At run-time, these learned policies are applied to incoming events/alerts to reduce false alert notifications. We validate our approach on two different datasets, log anomaly and metric anomaly events/alerts, to show percentage increase in accuracy over state-of-the-art methods by 7.39% and 35.7%, respectively.

KEYWORDS

Event Suppression, Alert Management, AIOps, Persistent Detection, Peak Detection

ACM Reference Format:
Karan Bhukar, Harshit Kumar, Seema Nagar, Pooja Aggarwal, Rohan Arora, Ruchi Mahindru, and Amit Paradkar. 2024. Dynamic Alert Suppression Policy for Noise Reduction in AIOps. In *46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24), April 14–20, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3639477.3639752

1 INTRODUCTION

Cloud computing's affordability comes with the complexity of loosely coupled micro-services, posing challenges for SREs in managing availability, performance, and capacity planning amidst vast amounts of machine-generated data [4, 13, 21]. Artificial Intelligence for IT Operations (AIOps) provides an AI-infused platform that empowers SREs with collaborative development and operations practices in an open, hybrid cloud environment [4] to manage applications/services running on cloud. AIOps platform mines voluminous amount of information from disparate data sources such as logs, metrics, traces, etc. to identify events[23] (log anomalies, metric anomalies), that are correlated and grouped together based on predefined or machine learned patterns[11], followed by fault localization [1], and use these to find similar historical incidents for action recommendation[16]. Events are de-duplicated and correlated by event correlation algorithms[11]. These algorithms, encompassing temporal, topological, and scope-based approaches, are employed to group related events into alerts. Subsequently, each generated alert triggers a notification to a SRE for addressal, serving as an alert mechanism to capture their attention and to enable timely resolution of the underlying issue.

It has been widely observed that often some of the alerts are short lived in nature[3, 8, 28], i.e., events associated with an alert may automatically resolve on their own. Hence, some of these alert notifications may not be worthy of SREs' attention, leading to false alarms. In the SRE's parlance, alert notifications that are false alarms are referred to as noise. As a result, SREs may ignore initial alert notifications to avoid frequent false alert notifications. However, the aforementioned approach may lead to overlooking crucial alert notifications and have serious consequences. It is conceivable that the deluge of false alert notifications could impede SREs' workflow, leading to decreased productivity. Prior attempts[30, 38] have focused on manually defining static policies applicable across all metrics or services. These polices act upon incoming events and keep the alert notifications suppressed until X events are observed over Y duration. In this paper, the policy consisting of X and Y is called as Alert Suppression Policy (*ASP*). Below we share a few challenges with manually defined static *ASP*: ***Dependency on Know-How:*** Manual policy creation is limited by SREs' experience and knowledge that was acquired through system observation. Such *ASPs* may therefore fail to consider all relevant dimensions while formulating, resulting in inadequate coverage and causing an increase in noise or false positive alert notifications. Moreover, one policy may not fit across all metric and services, because each metric or service may generate events with a different frequency and velocity. ***Dynamically changing environments:*** Manually defined *ASPs* may fail to adapt to the dynamic nature of the IT environment. For example, the same application stack running in a client's environment may exhibit different behavior on different



This work licensed under Creative Commons Attribution International 4.0 License.

ICSE-SEIP '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0501-4/24/04.
https://doi.org/10.1145/3639477.3639752

days due to changing workload patterns. Continuous updates are required to keep such policies aligned with the evolving system. ***Non-Scalable & High Maintenance:*** It is impractical to manually create multiple *ASPs* for a large volume of metric names and service names. Statistical approaches for data analytics are optimal for analyzing historical data and dynamic environments. Thus, it is imperative to leverage them for the generation, updating, and maintenance of *ASPs*. Thereby, relieving SREs from the burden of manual creation, activation, and deactivation of policies.

To address the aforementioned challenges, this work proposes a novel, scalable, and unsupervised method called *Dynamic-X-Y* for learning tailored *ASP* for each micro-service or metric. The method includes a Persistent Region Detection (PRD) module that utilizes historical alerts and its constituent event counts to identify peaks (high event count deviating from normal behavior) forming dense regions, which are then used to compute the X and Y values. These X, Y values are used to define *ASP*s, which are employed at run-time to suppress alerts that do not warrant SRE's attention, thereby reducing noise. The *PRD* module is based on the widely used statistical approach known as moving average envelop[2, 15] which has been applied in various domains, including finance[33, 34], economics[35], medical[26, 27]. The key contributions of this paper are: a) Propose a method that learns tailored *ASPs*, one for each metric or service based on its characteristics, such as velocity, peak and density. In the historical events and alert data, it identifies dense persistent regions where events occurred in huge quantities and use them to compute the values of X and Y for defining the dynamic *ASP*. (b) Evaluate the proposed method on different data modalities, including log anomaly events/alerts and metric anomaly events/ alerts. We compare the proposed method with the baselines *static-X-Y ASP* and *No-Suppression* (default approach) to show its superior performance. An important observation worth pointing out is that the dynamic *ASPs* learned through the proposed unsupervised method performs equivalent to the *ASPs* learned if the labeled data were available, i.e. supervised setting. (c) We perform a deeper analysis of the proposed method across several dimensions such as width-cutoff, window size, and density to provide key insights about how these features effect the quality of persistent regions. (d) We show the application of dynamic *ASPs* to demonstrate its usefulness in a real industrial setting.

2 METHOD

Before describing the method for learning *ASP*s, we first define the required preliminaries in AIOps for the ease of understanding of the reader.

2.1 Preliminaries

Cloud is designed to host multiple applications, each consisting of one or more micro-services that consume resources like CPU, memory, and network bandwidth. An application generate logs which is an essential piece of information required for issue diagnosis. Additionally, a metric associated with a resource is also another essential piece of information that is very helpful for issue diagnosis. A metric data associated with a resource is represented as time series of numerical values. A metric continuously emit values to communicate the state of the associated resource. For instance, an application running on the cloud may have two resources, *host-server1* and *host-server2*, with *host-server1* associated with two metrics, *cpu.utilization* and *memory.utilization*, while *host-server2* is associated with three metrics, *cpu.utilization*, *memory.utilization*, and *request.time*. This example produces two metric time series for *cpu.utilization*, two for *memory.utilization*, and one for *request.time*.

To formalize, we define a set of r resources as $\mathcal{R} = \{r_1, r_2, \ldots, r_r\}$ and a set of m items which can be either a metric or a service as $\mathcal{M} = \{m_1, m_2, \ldots, m_m\}$. Since each resource is associated with one or more items in M, the set of item-resource pairs can be partitioned into equivalence classes guided by the item, $[m_i] = R_i$, where $R_i \subset \mathcal{R}$. That is, each resource $r_{ij} \in R_i$ in an equivalent class $[m_i]$ is associated with the item name m_i. Each item-resource pair (m_i, r_{ij}) is a time series of real values of length T, represented as a real-valued vector $\mathbb{R}$. Also, each (m_i, r_{ij}) has an associated time series of anomaly events(Ω_{ij}) of same length T, where $\Omega(T) \in \{0, 1\}$, 1 represents an anomaly event and 0 represents no anomaly. For this work, any state-of-the-art anomaly detection algorithm[10] can be applied to (m_i, r_{ij}) time series for generating anomaly events time series Ω_{ij}. For the sake of brevity, the phrase anomaly event will be referred as event, and the events time series Ω_{ij} is referred to as Ω. The proposed approach, known as *Dynamic-X-Y* for alert suppression, comprises of the following three modules:

(1) **Persistent Regions Detection (PRD):** To identify regions with high frequency of events that are dense in the input historical events time series Ω.
(2) **X-Y Computation:** From the detected persistent regions, dynamically learn the number of events (X) and duration (Y) to automatically define tailored *ASP* per item.
(3) **Inference:** At run-time, apply the *ASPs* to suppress alerts that are not persistent and do not require SREs attention, hence reducing the noise.

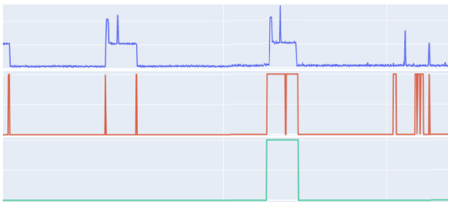


Figure 1: Three charts: *blue* for raw metrics, *red* for anomalous events, and *green* for persistent regions detected.

2.2 Persistent Regions Detection

A region in $\Omega(T)$ is identified as persistent if it contains a statistically significant number of events that are contiguous and dense over an extended period of time. Because each item (a service or a metric) behaves differently and has its own characteristics, hence, dynamic in nature, it may be the case that for an item, a persistent region consists of 10 events in 50 minutes. While for another item, a persistent region consists of 5 events in 50 minutes. The goal of *PRD* is to statistically learn one or more persistent regions for an item across all its resources. Each persistent region is characterized by four values: *Start*, *End*, *EventFrequency*, and *Duration*.

To understand it through an example, refer to Fig. 1, the x-axis represents time index. The first chart in *blue* color illustrates the raw metric values emitted from a resource. The second chart in *red* color shows the metric events time series (Ω) detected by a anomaly detection algorithm for the raw metric values in the chart above it. The input to the *PRD* module is an event time series (Ω), and the output is a list of persistent regions consisting of one or more metric anomalous events, shown in the third chart (in *green*). For example, Figure 1 illustrates that while there were multiple metric anomalous events, the *PRD* module identified a single persistent region. Figure 2 demonstrates the functioning of *PRD* module on a dummy event time series data, comprising of three steps:

- **Peak Detection:** identifies high ***frequency*** events window in time series Ω.
- **Dense Peak Detection:** identifies candidate persistent regions that consists of ***dense*** and ***contiguous*** peaks.
- **Merging and Selection:** identifies the final persistent region by merging the candidates from the previous step.

2.2.1 Peak Detection. The objective is to identify time windows in events time series Ω that deviate significantly from its moving average. We use a sliding window(SW_1) to capture the time-varying frequency of events in Ω. While traversing the time series Ω using the sliding window(SW_1) of *window size* W_1 and *window stride* S_1 (both measured in minutes), accumulate the count of events within each sliding window, represented as $\mathbb{H}$, refer Equation 1.

$$\mathbb{H}(i) = \sum_{j=i-(W_1-1)/2}^{i+(W_1-1)/2} \Omega(j) \tag{1}$$

To ensure that the size of $\mathbb{H}$ is same as Ω, a suitable length padding is used. To gain a better understanding of the application of SW_1(orange window), we direct the reader's attention to Figure 2 that vividly illustrates how SW_1 is employed to aggregate the occurrence frequency of events within time series Ω. Overall, the vector $\mathbb{H}$ captures the time-varying frequency of events in the time series Ω, with each entry representing the number of events observed within a fixed time window of duration W_1 with a *window stride* of S_1, i.e. $\mathbb{H}(T) \in \{0, 1, \ldots, W_1\}$. In this paper, the value of $W_1 = 20$ and $S_1 = 5$ (both in minutes) is used to calculate events' frequency.

In a cloud environment, an application failure triggers a surge of events, leading to a higher frequencies of events which is already captured in $\mathbb{H}$. The higher frequencies in $\mathbb{H}$ may deviate significantly from the mean frequency observed during normal system behavior. To identify higher frequencies in $\mathbb{H}$ that deviates from its mean, we use Bollinger Bands[19], a widely used moving average envelope method for analysis in financial domain. Bollinger Bands consist of three lines: a *moving average bollinger band (MABB)* (typically a 20-period simple moving average), an *upper bollinger band - UBB* (usually 2 standard deviations above the moving average), and a *lower bollinger band (LBB)* (usually 2 standard deviations below the moving average). These bands provide a measure of the volatility and fluctuation of an underlying time series. In this work, the UBB (equation 2) identifies high-frequency peaks in $\mathbb{H}$ that deviates from their moving averages.

$$\begin{aligned} \overline{MA}_i &= \frac{1}{N} \sum_{j=i-N}^{i-1} \mathbb{H}(j), \quad i \in 1, 2, \ldots, T \\ \mathbb{H}(i) &\geq \overline{MA}_i + \tau \cdot \sigma_i \end{aligned} \tag{2}$$

In Equation 2, at each index i in $\mathbb{H}$, the moving average over a period of length N is represented as $\overline{MA}_i$, the moving standard deviation is represented as σ_i, and τ is the multiplier. Bollinger Bands are well-suited for detecting high-frequency patterns or peaks in $\mathbb{H}$ because they provide a flexible and dynamic way of capturing volatility in the events frequency over a period of time.

$$\mathbb{F}(i) = \begin{cases} 1, & \text{if } \mathbb{H}(i) \geq \overline{MA}_i + \tau \cdot \sigma_i \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Equation 3 checks if the value of $\mathbb{H}(T)$ exceeds the moving average by a certain number of standard deviations (determined by the multiplier τ). This identifies data points in $\mathbb{H}$ that are significantly deviating from the overall trend, and therefore are likely to indicate *peaks*. The final output is a vector $\mathbb{F}$ of length T, where $\mathbb{F}(T) \in \{0, 1\}$. A value of 1 in $\mathbb{F}$ indicates that the events frequency within a period of length N is greater than or equal to the UBB of the corresponding time window in $\mathbb{H}$. And, a value of 0 in $\mathbb{F}$ indicates that the events frequency is below the UBB of the corresponding time window in $\mathbb{H}$. This output helps to identify time windows where event frequency peaked, exceeding the expected variability of the event time series. For an exemplary demonstration of the successful transformation of $\mathbb{H}$ into $\mathbb{F}$, see Figure 2. The values of $N = 40$ and $\tau = 3.5$ are empirically determined for detecting high-frequency peaks in $\mathbb{H}$.

2.2.2 Dense Peak Detection. Bollinger Bands identified high frequency peaks that deviates from their moving averages, and next we want to measure that the peaks detected in vector $\mathbb{F}$ are also contiguous. By traversing the time series $\mathbb{F}$ using a sliding window (SW_2) of size (W_2) and stride (S_2), accumulate peak signals present within each sliding window, represented as π, refer Equation 4.

$$\pi_i = \sum_{j=i-(W_2-1)/2}^{i+(W_2-1)/2} F(j) \tag{4}$$

If the accumulated count of peak signals for each index i in π is greater than the threshold D then it is a *dense* region indicated by a value of 1, else 0 to indicate a non-dense region, refer Equation 5.

$$\mathbb{K}(i) = \begin{cases} 1, & \text{if } \pi_i \geq D \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

These dense peak signals are candidate persistent regions, represented as vector $\mathbb{K}$, where $\mathbb{K}(T) \in \{0, 1\}$. The vector $\mathbb{K}$ captures the dense peaks present in vector $\mathbb{F}$, while filtering the isolated transient events. To gain a better understanding of the application of SW_2(red window) in detecting dense peaks within $\mathbb{F}$, see Figure 2. The value of $W_2 = 3$, $S_2 = 1$ and $D = 2$ are empirically determined for the purpose of detecting dense peaks in $\mathbb{F}$.

2.2.3 Merging and Selection. To ensure accurate identification of persistent regions, a two-step process is performed. Step 1 involves **merging of candidate persistent regions** in $\mathbb{K}$ that are in the neighboring vicinity of each other, and Step 2 involves **time-based selection of persistent regions**. Before performing the two steps,

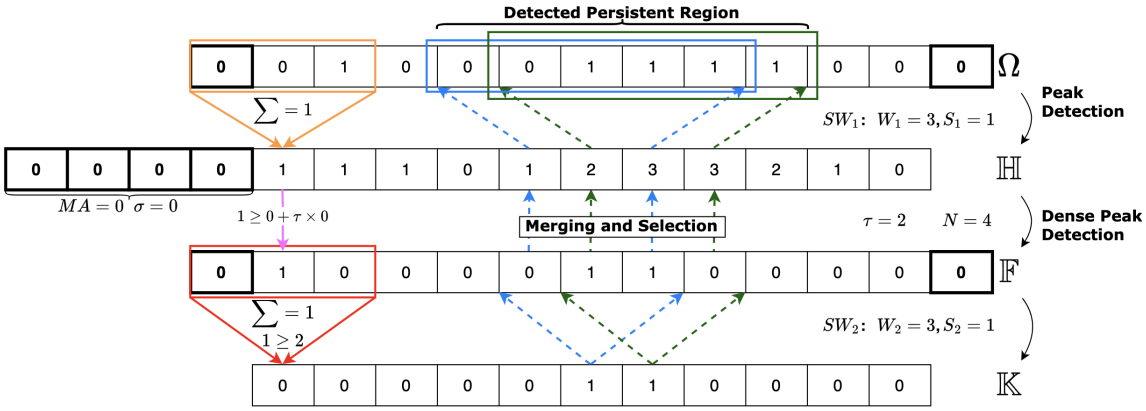


Figure 2: Running example to illustrate Persistent Regions Detection module on input event time series: bold-outlined 0's are padding blocks, bold arrows show first two sub-modules for computing $\mathbb{K}$ with Ω, dotted arrows depict sliding window backtracking and candidate persistent region detection.

the start and the end indexes of each candidate persistent region in $\mathbb{K}$ is mapped to its corresponding start and end timestamps of events in Ω. This mapping is achieved by backtracking the operations of two sliding windows, SW_1 and SW_2. The result is a list of candidate persistent regions $\mathcal{P}$. Figure 2 provides an illustration of the process by which the indices of the *dense peaks* present within $\mathbb{K}$ are backtracked to the corresponding timestamps of the candidate persistent regions residing in Ω. Each element in $\mathcal{P}$ has the following structure, $(Start_k, End_k, EventFrequency_k, Duration_k)$, where $Start_k$ and End_k represents the start timestamp and the end timestamp, and $EventFrequency_k$ is the events count within the duration $Duration_k$ of the candidate persistent region k. Each candidate persistent region in $\mathcal{P}$ is merged with its neighbouring candidate persistent region guided by the following heuristics. Regions A and B are considered neighbors if they are within ω minutes of each other. This step examines whether a persistent region is surrounded by other persistent regions in a nearby neighborhood and merges such neighboring regions into a single persistent region. Hence, creating a combined longer persistent region, instead of several small and isolated persistent regions. In the second step, persistent region is selected. If the duration of the merged candidate persistent region is longer than the β (width-cutoff) minutes, then it is classified as persistent region. This step helps filter out false positive candidate persistent regions from $\mathcal{P}$. After merging and selecting candidate persistent regions, the final set of persistent regions in $\mathcal{P}$ are used to compute the number of events (X) and the duration (Y) over which the events must be observed to unsuppress them. The parameters ω and β were set to 15 minutes.

2.3 X-Y Computation

The main objective of this module is to learn a value of X and Y for each equivalence class $[m_i] = R_i$, $\forall m_i \in \mathcal{M}$ and $R_i \subset \mathcal{R}$. The X and Y value for each item m_i is used to define a ASP that is applied at run-time for alert suppression. The X and Y value is central representative of the list of persistent regions in $\mathbb{P}_i$ for an item m_i, defined as follows in Equation 6.

$$\mathbb{P}_i = \bigoplus_{j=1}^{b} (\mathcal{P}_{i,ji} \cdot \mathbf{1}_{\{|\mathcal{P}_{i,ji}| > \alpha\}}) \tag{6}$$

where b is the number of resources in R_i and α is the minimum number of detected persistent regions required for each item-resource pair in $[m_i] = R_i$. That is, concatenate list of persistent regions across all b resources, where the number of persistent regions in the list $\mathcal{P}_{i,ji}$ associated with each resource $r_{ji} \in R_i$ is greater than α. For an item m_i, computing a central tendency value of X and Y from $\mathbb{P}_i$ is not a straightforward task because the persistent regions in $\mathbb{P}_i$ are not necessarily all of the same *Duration*. Hence, computing a mean or median directly of the *EventFrequency* present in $\mathbb{P}_i$ to get a central value of X may not be accurate. To address this, we first find the quantum persistent region $\mathcal{Q}$ in $\mathbb{P}_i$ which is defined as follows in Equation 7.

$$\begin{aligned} index &= \underset{q \in \mathbb{P}_i}{\text{argmin}}\{q.Duration\} \\ \mathcal{Q} &= \mathbb{P}_i[index] \end{aligned} \tag{7}$$

where $\mathcal{Q}$ is a persistent region in $\mathbb{P}_i$ with the smallest duration. Based on the duration in $\mathcal{Q}$, the *EventFrequency* is scaled down for each persistent region in $\mathbb{P}_i$ by the same ratio and stored in a vector $\mathbb{X}$, as shown in Equation 8:

$$\mathbb{X}(k) = \frac{\mathcal{Q}.Duration}{\mathbb{P}_i(k).Duration} \times \mathbb{P}_i(k).EventFrequency \tag{8}$$

The final values of X and Y are computed using Median($\mathbb{X}$) and $\mathcal{Q}.Duration$, respectively. The X and Y values for each item m_i are used to define the *ASP* which is used at run-time to suppress alerts, as explained below.

2.4 Inference

The ultimate goal of this work is to accurately determine when to suppress or unsuppress alerts using the learned *ASP*. At run-time, the observability tools continuously observe and collect the log data coming from a service and metric values coming from a metric. The anomaly detection algorithm acts upon these values to detect anomalous events which are then de-duplicated and

grouped together into alerts. As new events arrive, and if they are de-duplicated to an existing alert then the event count of the alert is updated until the issue is resolved. If the event count in an alert at any point in time exceeds the learned X value within Y duration as defined by the corresponding *ASP*, the alert is unsuppressed, leading to an alert notification to the SRE. Conversely, if the aforementioned condition is not met, the alert remains suppressed. The application of *ASP* minimizes unnecessary noise, thereby, enabling timely and accurate responses to critical alerts, facilitating the creation of actionable alert notifications for SREs.

3 EXPERIMENTAL SETUP

This section covers various aspects of the datasets used in the study, including its characteristics, the creation of annotated test datasets, the baseline methods, and evaluation metrics.

3.1 Datasets

In-order to validate the quality of persistent regions detection across different modalities of data i.e. logs and metrics, we collected log data for a conversation application deployed on cloud and the metric data was collected for a set virtual machines (VMs) hosting hundreds of applications deployed and provisioned on cloud. The metric dataset was prepared from 14 days of raw metric data, consisting of 52 metrics across 1836 resources. The total number of active metric-resource combinations were 3576. The metric anomaly detection algorithm detected 56, 442 anomalous events. Whereas, two years worth of log data was collected from a conversation application that consists of 15 micro-services, continuously generating log data as requests are made. The log anomaly detection algorithm identified 137, 269 log anomaly events. Not only the duration of log data is longer, it also contains a lot more anomalous events as compared to the metric data. An interesting characteristic of the log data that we would like to bring to readers attention is the inter-arrival frequency of events, i.e sproadic nature. For the metric data, the average duration of contiguous metric anomaly events is approximately $53(s.d.)$ minutes (a little less than hour) containing 11 events. Whereas, for the log data, the average duration of contiguous log anomaly events is approximately $87(s.d.)$ hours containing 2691 events. This suggests that the log events are less sporadic in nature as compared to the metric events. Or, whenever log events happen they happen for a larger duration of time, and metric events happen for a shorter duration of time.



Figure 3: Time series for a metric-resource pair showing persistent and non-persistent regions.

3.2 Test Dataset Preparation

In AIOps, alerts are unsuppressed by default, leading to an alert notification. As mentioned in the introduction section, SREs often complain that some of the alert notifications are false, and they unnecessarily spend considerable time sifting through numerous events to determine the underlying problem. Subsequently, false alert notifications may lead to lack of trust in the system. An effective alert suppression system has a direct correlation with its efficacy at detecting persistent regions and non-persistent regions. Note that, events in a persistent region will be unsuppressed, leading to an alert notification. Whereas, events in a non-persistent region will be suppressed, and hence no alert notification is generated. Therefore, to compare baselines (outlined in subsection 3.3) with our approach and to ensure accuracy and reliability, we need ground truth data of persistent and non-persistent regions. In order to generate accurate ground truth data, an expert in the field assigned labels to regions where anomalous events were observed keeping in mind the persistence characteristics, velocity and density. Only those time series were identified for human labeling that contains at least one anomalous events, resulting in 38 and 12 time series for metric and log data, respectively. The plot in Figure 3 shows 5 regions for an event time series. To receive ground truth labels from the annotator, we prepared a corresponding table (refer to Table 1) comprising of the *region number*, *events count per region*, *start time* and *end time* of the region. The last column in Table 1 shows human annotations, a *TRUE* value indicates a persistent region, while a *FALSE* value indicates a non-persistent region. Table 2 presents details and relevant statistics of the labeled test datasets for both metrics and logs.

Table 1: Table corresponding to regions in Figure 3 for labeling the last column as persistent (T) or not-persistent (F)

Region	Event	Start-Time	End-Time	Persistent
1	61	2022-08-29T04:30	2022-08-29T09:30	TRUE
2	20	2022-08-29T10:25	2022-08-29T12:00	TRUE
3	2	2022-09-07T07:00	2022-09-07T07:05	FALSE
4	3	2022-09-07T16:20	2022-09-07T16:30	FALSE
5	3	2022-09-07T23:40	2022-09-07T23:50	FALSE

Note that, just the mere presence of anomalous events in the time series does not guarantee persistent behavior, i.e. there could be a event time series that exhibits sporadic behavior. In such scenario, no persistent regions are observed, and therefore no ASP is generated. For item (either metric or microservice), where no policy is generated because persistent regions were not observed in the historical data, this indicates that the particular item was never effected when the fault happened in the system. Due to this reason, our algorithm was able to learns 16 *ASPs* one for each 16 metrics out of total 52 metrics in metric dataset, and similarly learns 12 *ASPs* one for each 12 micro-services out of total 15 micro-services in log dataset. For example, for the metric 'MemoryUsed', the Dynamic-X-Y method learned $X = 11$ and $Y = 55$ min. Similarly, in the log dataset, for the 'NLU' service, the proposed method learned $X = 4512$ and $Y = 12.5$ hours. These results very well align with the less sporadic nature of log data as mentioned in Section 3.1.

3.3 Baseline Methods and Proposed Method

To evaluate the performance of our proposed method for alert suppression, we compare it with the two baseline methods.

Table 2: Statistics of annotated Test Datasets

	Metric Dataset	Log Dataset
Number of persistent regions	111	32
Number of non persistent regions	345	42
Number of metric/service names	12	12
Number of resource names	36	NA
Number of events	4225	137269
Total Number of samples	38	12

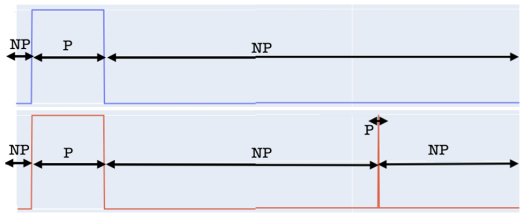


Figure 4: Blue and Red graphs: Ground Truth and Prediction

(1) **No-Suppression**: This method creates an alert notification if an anomalous event is detected. That is, any region in an event time series Ω having at least one anomalous event is a persistent region. This is equivalent to $X = 1$ in an *X-Y-Policy* which is also the default behavior.
(2) **Static-X-Y**: This method uses predefined static X and Y values for detecting persistent regions, suggested by a domain expert. While one could determine the best X and Y values for a dataset using ground truth (if available) and a brute force search, it is usually not feasible due to unavailability of ground truth data. Domain experts are consulted instead, but their suggested values may not be sufficient for dynamically changing scenarios (discussed earlier). If 6 or more events are reported in 30 minutes, the corresponding region is persistent, and the alert is unsuppressed. The static X and Y values for this setting are 6 and 30, respectively.
(3) **Dynamic-X-Y**: Our proposed method dynamically learns X and Y values from historical data for each item in an unsupervised setting, i.e. no ground truth is needed to learn the values of X and Y. One clear benefit is that it observes and *recommends a tailored X-Y-Policy for* ***each item***, unlike the aforementioned baseline method that has a *static X-Y-Policy applied across* ***all metrics or services***.

3.4 Evaluation Metrics

During evaluation, each of the aforementioned methods detected persistent regions in event time series to unsuppress alerts. To evaluate the accuracy of methods used for detecting persistent and non-persistent regions, standard evaluation metrics such as True-Positive, False-Positive, True-Negative, and False-Negative are utilized to calculate F1 score [31] and Accuracy [31]. True-Positive measures if a method detected a ground-truth *persistent region as a persistent region*, while False-Positive measures if a method detected a ground-truth *non-persistent region as a persistent region*, True-Negative measures if a method detected a ground-truth *non-persistent region as a non-persistent region*, while False-Negative measures if a method detected a ground truth *persistent region as a non-persistent region*. Figure 4 shows two plots for a time series data; the top plot is the manually curated ground truth, and the bottom plot is the output of the *PRD* module showing detected persistent region(P) and non-persistent region(NP). It detected two persistent regions and three non-persistent regions, whereas the ground truth indicate only one persistent region and three non-persistent regions, therefore $TP = 1$ and $FP = 1$, $TN = 3$ and $FN = 0$.

4 EXPERIMENTS

We address the following questions through our experiments:

(1) How does the performance of the proposed method compare to the baseline methods?
(2) How does the proposed unsupervised method Dynamic-X-Y compares to the X and Y values computed using a supervised setting?
(3) What are the effects of the hyper-parameters on the performance of the proposed method?

4.1 Performance Analysis

Table 3 compares the performance of our proposed method with the baselines. The results show that our method *Dynamic-X-Y* outperforms both the baselines by significant margin. On the metrics dataset, the percentage improvement in accuracy of persistent regions detection is 45.8% (64.41 to 93.93) and 7.4% (87.46 to 93.93) compared to the *No-Suppression* and *Static-X-Y* methods, respectively. Similarly, on the Log dataset, the percentage improvement in accuracy is 37.5% (50.23 to 69.09) and 35.7% (50.90 to 69.09) compared to the same methods. The reason why both baseline methods exhibit lower F1 and Accuracy scores than the *Dynamic-X-Y* approach on the two test datasets, is because they are designed to have greater sensitivity for detecting persistent regions. Consequently, these methods prioritize a wider coverage of alert notifications, including both persistent and non-persistent regions, with the trade-off of potentially generating a higher number of false positive persistent regions. For example, the *No-Suppression* method identifies a region as persistent if at least one anomalous event is observed. Thus detecting numerous false positive persistent regions, and hence resulting in a lower F1 and Accuracy score. It generates alerts for nearly every detected region, leading to an excessive number of noisy alerts that can hinder the SREs' ability to focus on alerts that actually require their attention, ultimately resulting in inefficiencies. The *Static-X-Y* method, on the other hand, is more stringent than the *No-Suppression* method, calling a region persistent only when it observes at least X=6 anomalous events within a Y= 30-minute interval. As a result, *Static-X-Y* detects fewer persistent regions, also leading to reduction in the number of false positive persistent regions. This improvement is evident in both F1 and Accuracy scores when compared to the *No-Suppression* method on the Metric dataset. It's worth noting that in the case of the Log Dataset, the performance of *Static-X-Y* method is very similar to the *No-Suppression* method. This can be attributed to the fact that the average number of events in each persistent region is 2691 in log dataset, which is greater than the X= 6 of *Static-X-Y* method and

also greater than X= 1 of *No-Suppression* method. Consequently, in the log dataset, the *Static-X-Y* method detects a similar number of persistent regions as *No-Suppression* method. This suggests that the *Static-X-Y* method's parameters, X and Y, are not domain agnostic, and its efficacy may vary with new data from different domains having different characteristics in terms of velocity and density of anomalous events. In contrast, the proposed *Dynamic-X-Y* method adapts by learning optimal X and Y values for each dataset. For example, in the log dataset, the average X value learned from all micro-services is 2376. This value is comparable to the average number of events (which is 2691) in each persistent region within the log dataset. This adaptation seeks to strike a balance in detecting true positive persistent regions while minimizing the false positive persistent regions, as reflected in the state-of-the-art F1-Score and Accuracy scores presented in Table 3.

Table 3: Model Comparison on Test datasets

	Log Dataset		Metric Dataset	
Method	**F1**	**Acc.**	**F1**	**Acc.**
No-Suppression	64.89	50.23	43.20	64.41
Static-X-Y	64.70	50.90	83.90	87.46
Dynamic-X-Y(Our)	**66.00**	**69.09**	**92.61**	**93.93**

4.2 Unsupervised vs Supervised setting

For this subsection and the following subsection, we will be using metric data for our study. This is because events in metric data exhibit sporadic behavior, it is imperative that greater benefits are observed when tailored alert suppression policies are applied.

Table 4: *ASP*s Learned by *Dynamic-X-Y* method

Metric	X	Y	Metric	X	Y	Metric	X	Y
MemResident	11	55	CpuUsage	6	60	CpuSys	11	55
TotalIOTime	12	60	HttpError	4	50	CpuUsed	10	50
RequestTime	8	55	WaitTime	11	55	LoadMin	9	55
MemoryUsed	11	55	TcpResets	6	50	ReqTime	10	50
FSWriteBytes	10	55	TcpRetrans	10	50	-	-	-
CounterTxt	7	60	CpuUser1	9	50	-	-	-

In a scenario with labeled data available, we perform a brute force search to find the optimal values for X and Y that maximize Accuracy in detecting persistent regions. We then compare the results of this search to our unsupervised approach, which does not require labeled data. Comparing the accuracy of the optimal values to the *Dynamic-X-Y* method will assess its capability and robustness. The experiment utilizes the metric dataset mentioned in Section 3. Since events are arriving every 5 minutes, the arrival of i events will take a minimum of $i * 5$ minutes. Thus, for $X = i$, the potential values of Y fall within the range of $\{(i) \times 5, (i+1) \times 5, \ldots, 11 \times 5, 12 \times 5\}$, where $2 \leq i \leq 12$. For instance, if $X = 11$, then the possible range of Y would be $\{55, 60\}$. This results in a total of 66 X-Y pairs. Figure 5 shows the 3-dimension plot of these 66 combinations, where x-axis represents the number of events (X), y-axis represents the time interval (Y) in steps of 5 minutes, and z-axis represents the accuracy score for each X-Y pair. The accuracy keeps increasing as X keeps increasing, and it reaches a maximum accuracy of 94.42 at $(X, Y) = \{(9, 45), (9, 50), (9, 55), (9, 60)\}$. After which, accuracy drops as X keeps increasing. This means the optimal value of X and Y using brute force approach is 9 and 50, respectively. *Dynamic-X-Y* approach learns an optimal X and Y value for each metric name. , refer Table 4. To compare with the single optimal X and Y value computed using brute force search under supervised settings, we take the mean of the X and Y values across all metric names in Table 4 learned by *Dynamic-X-Y*. This approach allows us to learn a single value of X and Y that is valid across all metric names for *Dynamic-X-Y*. The calculated mean of X and Y values over all metrics are 9.06 and 54.06, respectively. It is worth noting that the learned mean value of X and Y from the unsupervised approach is very close to the optimal X and Y values found through the brute force search under supervised setting. Determining X and Y values under supervised setting is a hypothetical approach, since labeled data is hard to get. The values of X and Y under the supervised setting is the most idealistic value, and therefore serves as an upper bound on how good a model can perform. The ability of the *Dynamic-X-Y* method to determine the best value of X and Y under unsupervised setting shows the overall strength of the method. **This shows that the *Dynamic-X-Y* approach can learn optimal *X* and *Y* values in the absence of labeled data**.

4.3 Ablation Study

Through ablation studies, this subsection assesses the impact of three hyper-parameters on the effectiveness of the *Dynamic-X-Y* method: Width-Cutoff (β), Window-Size (W_1), and Count of Persistent Region (α). For each ablation study, the parameter under investigation is varied while all other parameters are kept constant.

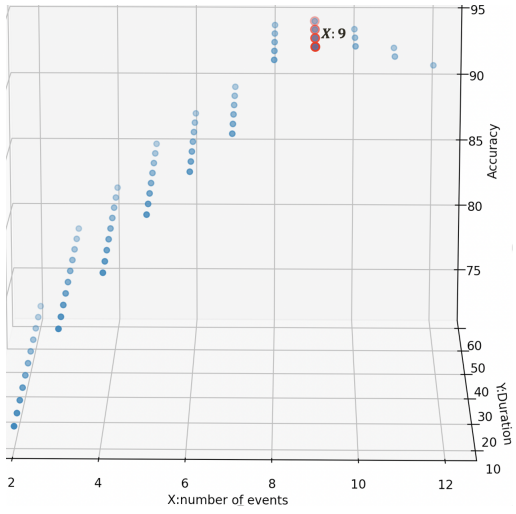


Figure 5: Performance of 66 possible Static-X-Y policies on the test dataset

4.3.1 Effect of Width-Cutoff(β). As mentioned in subsection 2.2.3, the Width-Cutoff parameter is used to select persistent regions from a pool of candidate persistent regions that have duration exceeding β minutes. Table 5 illustrates the effect of varying β on

Table 5: Effect of parameter Width-Cutoff(β) on the performance of the *Dynamic-X-Y* approach

Width-Cutoff(β)	Policies	P	R	F1	Acc
10	21	61.49	**94.28**	74.43	81.26
15	16	**95.92**	89.52	**92.61**	**93.93**
20	12	78.95	87.37	82.95	87.02
25	10	73.17	87.37	79.64	84.82
30	10	73.17	87.37	79.64	84.82
35	9	72.95	86.41	79.11	84.48

the performance of the *Dynamic-X-Y* approach. A *higher value of β implies a stricter* observation of candidate persistent regions by the algorithm, while a lower value of β leads to the opposite effect. A lower value of β results in the filtering of relatively fewer candidate persistent regions, leading to the selection of more candidate persistent regions as final persistent regions. In Table 5, when $\beta = 10$, there are 21 policies, whereas when $\beta = 15$ there are 16 policies. The persistent regions are detected for a greater number of metrics and relatively more number of persistent regions for each metric. These regions will likely have a smaller size, resulting in smaller X values. Policies with smaller X values will lead to larger number of irrelevant alerts, thereby defeating the objective of noise reduction. In summary, *low value of β results in learning more policies*, each policy has low value of X and Y, triggering false alert notifications. Exactly opposite will occur when we increase the value of β; Less persistent regions are detected for a reduced set of metrics and hence less policies, each persistent region of relatively larger size with higher value of X. Learning less policies with higher values of X and Y is also not desirable, because for metrics where no policies are available, the *No-Suppression* default method is used which will also lead to false alert notifications, and hence low precision. This is the reason why precision drops as the value of parameter width-cutoff (β) increases. **These results indicate that there is an optimal value of β that should be used for identifying persistent regions to learn values of X and Y. Both high or low values of β leads to more false positives which in turn reduces precision, leading to a drop in accuracy.** For this dataset, the most optimal value of parameter Width-Cutoff (β) is 15, leading to 16 policies.

Table 6: Effect of parameter Window-Size(W_1) on the performance of the *Dynamic-X-Y* approach

W_1	Policies	P	R	F1	Acc.
10	16	**95.92**	89.52	**92.61**	**93.93**
15	16	84.95	91.43	88.07	90.48
20	17	70.71	**94.28**	80.81	85.35
25	19	57.22	93.39	70.96	79.23
30	25	33.22	92.79	48.93	67.37

4.3.2 Effect of Window-Size(W_1). The parameter W_1 is defined in the peak detection (subsection 2.2.1) to control the length of the sliding window (SW_1) over which events are accumulated, resulting in a time series ($\mathbb{H}$). Table 6 presents the effect of varying the parameter W_1 on the performance of the *Dynamic-X-Y* approach. ***One observation is that there is a direct proportionality between the Window-Size(W_1) and the number of policies learned.*** In other words, the *Dynamic-X-Y* method learns more policies when the value of W_1 is increased. ***Another key observation is that while the method learns more X-Y-Policies, the corresponding X value in those policies gets smaller with an increase in W_1.*** For instance, the metric Mem.Resident has the values of $X = \{11, 10, 8, 8, 8\}$ when the value of $W_1 = \{10, 15, 20, 25, 30\}$. To gain insight into this behavior, let us consider the example depicted in Figure 6. Here, we observe two sliding windows of different sizes (3 and 9) traversing across the same input event series Ω to generate the corresponding $\mathbb{H}$. In Ω, there exist two regions containing events, where the first region contains only one event while the second region contains six events. Irrespective of the window size, the second region is always detected as a persistent region. Whereas, the first region is detected as a persistent region when the window size is 9. The reason for this is that when the window size is 3, the signal at index 2 in $\mathbb{H}$ fails to meet the UBB condition ($1 \geq MA + \tau \times \sigma$, where $MA = 0.25$, $\sigma = 0.43$). Therefore, it is undetected as a peak, so a persistent region is not detected which is a true negative. However, when the window size is increased to 9, the event frequency increases from 1 to 2, and the signal at index 2 is detected as a peak because it satisfies the UBB condition ($2 \geq MA + \tau \times \sigma$, where $MA = 0.25$, $\sigma = 0.43$), leading to the detection of the persistent region, which is a false positive. Consequently, with $W_1 = 3$, the X value is equal to the *EventFrequency* of the second region which is detected as a persistent region. Whereas, with $W_1 = 9$, the X value is equal to the median of the *EventFrequency* of the first and second region which is obviously less than the X value calculated with $W_1 = 3$. In summary, when the window size W_1 is increased, the events in Ω tend to interfere with each other during windowing for computing vector $\mathbb{H}$. As a result, candidate persistent regions are detected with a smaller *EventFrequency*, thereby lowering the overall median (final X), leading to more number of policies. During inferencing, these policies may generate more false

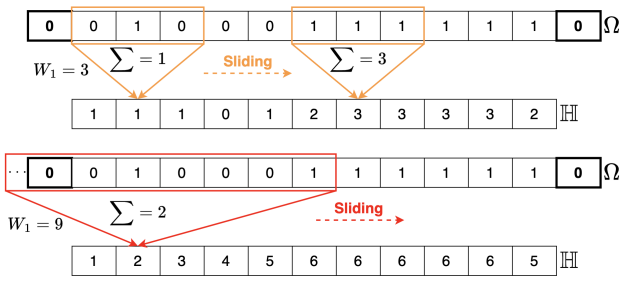


Figure 6: Illustrating the effect of Window-Size(W_1)

alert notifications, especially when the learned value of X is small. Based on the above discussion, it is evident that setting $W_1 = 30$ will result in more policies with smaller values of X, leading to false alert notifications, and hence lower precision and accuracy of 33.22 and 67.37, respectively (Table 6). In conclusion, the Window-Size (W_1) has a direct relationship with the number of false alert notifications, and therefore the accuracy decreases as the W_1 increases.

4.3.3 Effect of Number of Persistent Regions (α). The parameter α is defined in the subsection 2.2.3. It suggests how many minimum number of persistent regions must exist in the list of final persistent regions $\mathcal{P}_{i,ji}$ for calculating the values of X and Y. Table 7 illustrates the effect of varying the value of α on the performance of the *Dynamic-X-Y* approach. For this experiment, all other hyperparameters were constant while varying the value of the parameter α. As α increases, the majority of metric names with zero or one persistent region are filtered out by the threshold imposed on $\mathcal{P}_{i,ji}$, refer Equation 6. Consequently, no *X-Y-Policy* is learned for most of these metric names. For instance, when $\alpha = 1$, only 16 out of 52 total metric names have *X-Y-Policies* learned, whereas when $\alpha = 12$ the number of *X-Y-Policies* are 0. If *X-Y-Policy* is not available for a metric name at run-time, we apply the *No-Suppression* approach which will have higher false positives leading to lower precision. As α approaches 12, the *Dynamic-X-Y* is not able to learn *X-Y-Policy* for any metric name and the results become exactly same as the *No-Suppression* baseline result as shown in Table 7. For this dataset, when $\alpha = 1$, the number of policies are 16. The value of $\alpha = 0$ is an oxymoron in this case, X and Y values cannot be learned if there are no persistent regions. These results suggest that **while designing an alert suppression system, the algorithm can learn *X* and *Y* values when at least one persistent region is detected**.

Table 7: Effect of the count of persistent regions(α) on the performance of the *Dynamic-X-Y* approach

Count(α)	Policies	P	R	F1	Acc.
1	16	**95.92**	89.52	**92.61**	**93.93**
3	8	66.89	**93.39**	77.95	83.38
5	5	35.86	88.88	51.11	67.90
7	2	33.02	92.31	48.64	65.09
9	1	29.59	92.31	44.81	64.23
11	1	29.59	92.31	44.81	64.23
12	0	28.20	92.31	43.20	64.41

5 INDUSTRIAL CASE STUDY

Next, we present key insights and findings from the industrial case study, derived from applying the proposed methodology to a real-world production dataset.

5.1 Learn and Apply Dynamic-X-Y Method

We present a tool that utilizes the *Dynamic-X-Y* method to learn and apply *X-Y-Policies*. Figure 7 displays the user interface of the training tool. By clicking the hyperlink "Train models" on the top left, the *Dynamic-X-Y* algorithm retrieves historical alert data from the alert database to learn the tailored *ASPs*. Once the policies are learned, they are sent to an automation dashboard, as shown in Figure 8. In the automation dashboard, a user can enable or disable suppression policies in real-time. As an exemplar, only 3 out of 16 policies are shown. When the policy is enabled, it acts upon an incoming alert to suppress or unsuppress it. Clicking on a policy name in the *Automations Dashboard UI* provides detailed information about the corresponding policy.

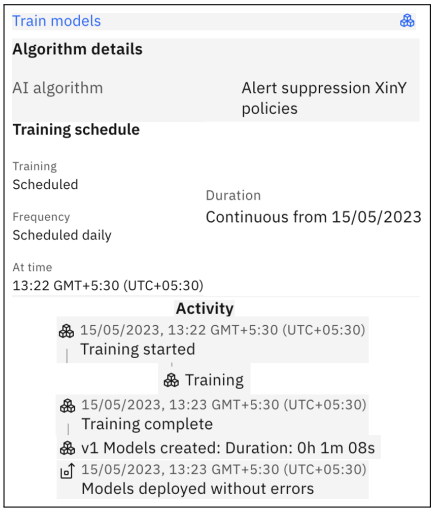


Figure 7: Tool UI to learn X-Y Policies using *Dynamic-X-Y*

5.2 Demonstrate Efficacy of X and Y Policies

The focus of this sub-section is to demonstrate the practical application and effectiveness of the ASPs learned using the novel *Dynamic-X-Y* method. Specifically, we examine its efficacy in noise reduction by suppressing alerts for a specific metric called "TcpRetrans". The data used in this study spans over 14 days, and consists of 206 anomalous events. As illustrated in the *Red* plot of Figure 9, these events are plotted against their respective timestamps, offering a clear visual representation of their distribution. Figure 9 shows 206 anomalous events grouped temporally as described in Section 1, leading to 13 distinct alerts, the number adjacent to each alert represents the event count. Each peak in the *Red* plot represents an alert. Using the *No-Suppression* baseline, each alert leads to

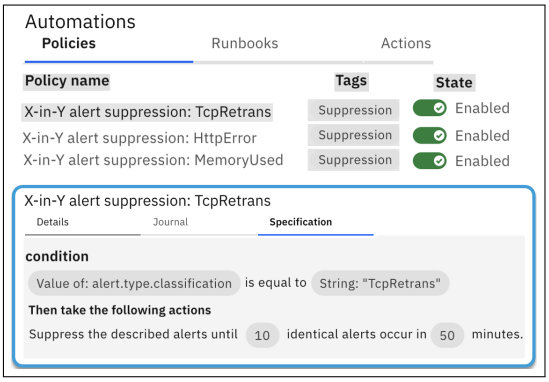


Figure 8: Automations Dashboard UI with X-Y Policies

an alert notification. Hence, 13 alerts are notified to the SRE but not all alerts in the *Red* plot are persistent. Some of these alerts have a low *EventCount* and are transient in nature, thus, resolve on their own without necessitating intervention from the SRE. Utilizing historical training data for the "TcpRetrans" metric, our *Dynamic-X-Y* method learns an *ASP* with $X = 10$ and $Y = 50$ (Table 4). Upon

applying this ASP, all alerts with an $EventFrequency \leq 10$ within a 50-minute window are suppressed, significantly reducing the number of alerts requiring SREs' attention. Consequently, only 5 alerts remain unsuppressed, as depicted in the *Green* plot of Figure 9. When employing the *Dynamic-X-Y*(Green plot) method, SREs are notified only of these 5 alerts, as opposed to the 13 and 9 alerts generated by the *No-Suppression*(Red plot) and *Static-X-Y*(Orange plot) method, respectively. Less alert notifications allow SREs to focus on real and critical issues, thereby improving overall efficiency and reducing MTTD and MTTR an issue. Compared to the *No-Suppression* and *Static-X-Y* methods, the *Dynamic-X-Y* method achieves a remarkable reduction of 61.53% (from 13 to 5) and 44.44% (from 9 to 5) in alert noise, respectively. This reduction in noise is achieved without compromising the detection of actionable alerts. The remarkable reduction in alert noise highlights the ability of the ***Dynamic-X-Y*** method to improve alert management and streamline the workflow of SREs in practical scenarios. Additionally, event

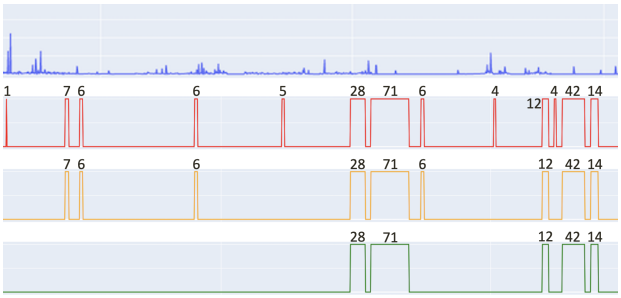


Figure 9: For the metric TcpRetrans, the four plots from top to bottom represent raw metric data (Blue), 13 alerts using No-Suppression (Red), 9 alerts using Static-X-Y (Orange), and 5 alerts using *Dynamic-X-Y* method (Green)

noise is also reduced alongside alert noise. For instance, in the case of *No-Suppression*, *Static-X-Y*, and *Dynamic-X-Y* methods, SREs need to review a total of 206, 192, and 167 events, respectively. This translates to a reduction of 18.93% and 13.02% in event noise when compared to the *No-Suppression* and *Static-X-Y* methods, respectively. ***Dynamic-X-Y* method effectively prioritizes alerts requiring SRE's attention** in addressing issues arising in client's environment with agility.

6 RELATED WORK

Alert notification plays a crucial role in ensuring timely human intervention in response to cloud anomalies, which can significantly impact both cloud reliability and the business revenue of cloud providers. However, on-call Site Reliability Engineers (SREs) often face challenges in locating and resolving faulty cloud services, primarily due to the prevalence of misleading, uninformative, and non-actionable alerts. This problem becomes even more complex when systems undergo sporadic changes and upgrades, as observed in the growing trend of continuous deployment.

Research in Alert Management: Numerous efforts have been made in the field of alert management, including aggregation [24], correlation [25, 29], and clustering [20, 37, 38]. However, existing approaches still struggle to effectively identify severe actionable and persistent alerts amidst the overwhelming number of misleading and non-actionable alerts. Threshold-based methods [36] have been previously employed but have proven insufficient in dealing with evolving systems. Although adaptive monitoring, which dynamically adjusts monitoring thresholds, has been explored [17, 18], these methods often suffer from false positive anomalies. Recently, machine learning techniques have been explored for alert management. Du et al.[7] proposed a deep learning-based approach for log analysis, aiming to identify anomalies and generate alerts. However, this method fails to suppress irrelevant alerts and is also prone to false positive anomalies. Another recent study by Zhao et al. [39] proposed a method that utilizes textual features and anomaly detection to identify severe alerts in monitoring metrics.

In addition to the aforementioned related work, **Research on peak detection algorithms** have found application in various domains, such as bioinformatics [5], mass spectrometry [5], signal processing [9, 14], image processing [22], and astrophysics [40]. However, to the best of our knowledge, our paper is the first to introduce the moving average envelope-based peak detection module followed by a dense peak detection module that efficiently capture persistent peaks over significant time periods. As a result, our method can significantly reduce the number of false alerts, improving the overall performance of alert management systems.

Industry Tools for alert suppression: Various industry tools have incorporated alert suppression capabilities to improve the management of alerts. Splunk [32], a popular AIOps tool, offers suppression groups that allow users to define a suppression period for similar alerts. Similarly, Datadog [6] provides a downtime API, enabling users to specify a period during which no alerts will be displayed. However, this API is more suitable for scheduled maintenance or testing scenarios rather than day-to-day monitoring of cloud applications. Another AIOps tool, Instana [12], introduces Smart alerts, which allows to define rule based alerts with configurable threshold for alert suppression. Despite these features, these existing products lack the ability to dynamically generate alert suppression policies by efficiently utilizing historical data. Our unsupervised method dynamically learns X-Y policies by efficiently detecting persistent regions from historical events and alerts data.

7 CONCLUSION

SREs are often overwhelmed by the huge volume of false alert notifications, leading to lower productivity, inefficiencies, and SLO violations. To address this limitation, this paper proposes an approach, called *Dynamic-X-Y*,. It is based on an unsupervised statistical method that analyzes historical event timeseries data to identify persistent regions for learning custom tailored alert suppression policies for for each metric or resource. At run-time, as events arrive and alerts are formed, these policies act upon them to identify if the events are persistent and whether they should be suppressed or not. This results in fewer but more relevant alert notifications, and hence less noise, enabling faster incident diagnosis and resolution. To demonstrate the effectiveness of our method, we prepared two test datasets, one containing logs collected from a conversation application and the other containing metric data collected from a private cloud. Experiment results highlight that the proposed method outperforms the baselines for both datasets.

REFERENCES

[1] Pooja Aggarwal, Seema Nagar, Ajay Gupta, Larisa Shwartz, Prateeti Mohapatra, Qing Wang, Amit Paradkar, and Atri Mandal. 2021. Causal Modeling based Fault Localization in Cloud Systems using Golden Signals. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 124–135. https://doi.org/10.1109/CLOUD53861.2021.00026

[2] L. Borsellino and P. Crisafulli. 2001. *The Day Trader's Course Workbook: Step-by-step exercises to help you master The Day Trader's Course*. Wiley. https://books.google.co.in/books?id=1pDhwAEACAAJ

[3] Mainak Chakraborty and Ajit Pratap Kundan. 2021. Architecture of a Modern Monitoring System. In *Monitoring Cloud-Native Applications: Lead Agile Operations Confidently Using Open Source Software*. Springer, 55–96.

[4] Junjie Chen, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. An empirical investigation of incident triage for online service systems. In *ICSE-SEIP 2019*. IEEE, 111–120. https://doi.org/10.1109/ICSE-SEIP.2019.00020

[5] Kevin R Coombes, Spiridon Tsavachidis, Jeffrey S Morris, Keith A Baggerly, Mien-Chie Hung, and Henry M Kuerer. 2005. Improved peak detection and quantification of mass spectrometry data acquired from surface-enhanced laser desorption and ionization by denoising spectra with the undecimated discrete wavelet transform. *Proteomics* 5, 16 (2005), 4107–4117.

[6] Datadog, 2023. Suppress Alerts with Downtimes. Retrieved Accessed: May 1, 2023 from https://docs.datadoghq.com/monitors/guide/suppress-alert-with-downtimes/?tab=api

[7] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[8] Dipanwita Guhathakurta, Pooja Aggarwal, Seema Nagar, Rohan Arora, and Bing Zhou. 2022. Utilizing Persistence for Post Facto Suppression of Invalid Anomalies Using System Logs. (2022).

[9] K. Harmer, G. Howells, W. Sheng, M. Fairhurst, and F. Deravi. 2008. A Peak-Trough Detection Algorithm Based on Momentum. In *2008 Congress on Image and Signal Processing*, Vol. 4. 454–458. https://doi.org/10.1109/CISP.2008.704

[10] Alexis Huet, Jose Manuel Navarro, and Dario Rossi. 2022. Local Evaluation of Time Series Anomaly Detection Algorithms. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 635–645.

[11] Jinho Hwang, Larisa Shwartz, Qing Wang, Raghav Batta, Harshit Kumar, and Michael Nidd. 2021. FIXME: Enhance software reliability with hybrid approaches in cloud. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 228–237.

[12] Instana, 2023. IBM Instana Observability: Smart Alerts. Retrieved Accessed: May 1, 2023 from https://www.ibm.com/docs/en/instana-observability/current?topic=websites-smart-alerts

[13] Jiajun Jiang, Weihai Lu, Junjie Chen, Qingwei Lin, Pu Zhao, Yu Kang, Hongyu Zhang, Yingfei Xiong, Feng Gao, Zhangwei Xu, et al. 2020. How to mitigate the incident? an effective troubleshooting guide recommendation technique for online service systems. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1410–1420. https://doi.org/10.1145/3368089.3417054

[14] V.T. Jordanov and D.L. Hall. 2002. Digital peak detector with noise threshold. In *2002 IEEE Nuclear Science Symposium Conference Record*, Vol. 1. 140–142 vol.1. https://doi.org/10.1109/NSSMIC.2002.1239285

[15] C.D. Kirkpatrick and J.A. Dahlquist. 2010. *Technical Analysis: The Complete Resource for Financial Market Technicians*. Pearson Education. https://books.google.co.in/books?id=I5SgX5q5sQEC

[16] Harshit Kumar, Ruchi Mahindru, and Debanjana Kar. 2022. Metadata-based retrieval for resolution recommendation in AIOps. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1379–1389.

[17] Seungchul Lee, Lin Li, and Jun Ni. 2010. Online degradation assessment and adaptive fault detection using modified hidden Markov model. *Journal of Manufacturing Science and Engineering* 132, 2 (2010).

[18] Andre Lemos, Walmir Caminhas, and Fernando Gomide. 2013. Adaptive fault detection and diagnosis using an evolving fuzzy classifier. *Information Sciences* 220 (2013), 64–85.

[19] Joseph Man-Joe Leung and Terence Tai-Leung Chong. 2003. An empirical comparison of moving average envelopes and Bollinger Bands. *Applied Economics Letters* 10, 6 (2003), 339–341.

[20] Derek Lin, Rashmi Raghu, Vivek Ramamurthy, Jin Yu, Regunathan Radhakrishnan, and Joseph Fernandez. 2014. Unveiling clusters of events for alert and incident management in large-scale enterprise it. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1630–1639.

[21] Yi Liu, Shujie Han, Cheng He, Jiongzhou Liu, Fan Xu, Tao Huang, and Patrick P. C. Lee. 2020. An Introduction to PAKDD CUP 2020 Dataset. In *Large-Scale Disk Failure Prediction*, Cheng He, Mengling Feng, Patrick P. C. Lee, Pinghui Wang, Shujie Han, and Yi Liu (Eds.). Springer Singapore, Singapore, 1–11. https://doi.org/10.1007/978-981-15-7749-9_1

[22] Meng Ma, Arjan Van Genderen, and Peter Beukelman. 2005. Developing and Implementing Peak Detection for Real-Time Image Registration. (01 2005).

[23] Ruchi Mahindru, Harshit Kumar, and Sahil Bansal. 2021. Log Anomaly to Resolution: AI Based Proactive Incident Remediation. In *36th IEEE/ACM International Conference on Automated Software Engineering (ASE-2021)*. IEEE/Association for Computational Linguistics, Melbourne, Australia.

[24] Dapeng Man, Wu Yang, Wei Wang, and Shichang Xuan. 2012. An alert aggregation algorithm based on iterative self-organization. *Procedia Engineering* 29 (2012), 3033–3038.

[25] Seyed Ali Mirheidari, Sajjad Arshad, and Rasool Jalili. 2013. Alert correlation algorithms: A survey and taxonomy. In *Cyberspace Safety and Security: 5th International Symposium, CSS 2013, Zhangjiajie, China, November 13-15, 2013, Proceedings 5*. Springer, 183–197.

[26] Jeong-Seon Park, Sang-Woong Lee, Unsang Park, et al. 2017. R peak detection method using wavelet transform and modified shannon energy envelope. *Journal of healthcare engineering* 2017 (2017).

[27] M Rakshit, D Panigrahy, and PK Sahu. 2016. An improved method for R-peak detection by using Shannon energy envelope. *Sādhanā* 41 (2016), 469–477.

[28] Navin Sabharwal and Gaurav Bhardwaj. 2022. What Is AIOps? In *Hands-on AIOps: Best Practices Guide to Implementing AIOps*. Springer, 1–17.

[29] Saeed Salah, Gabriel Maciá-Fernández, and Jesús E Díaz-Verdejo. 2013. A model-based survey of alert correlation techniques. *Computer Networks* 57, 5 (2013), 1289–1317.

[30] Moon Sun Shin and Kyeong Ja Jeong. 2006. Alert correlation analysis in intrusion detection. In *Advanced Data Mining and Applications: Second International Conference, ADMA 2006, Xi'an, China, August 14-16, 2006 Proceedings 2*. Springer, 1049–1056.

[31] Marina Sokolova and Guy Lapalme. 2009. A systematic analysis of performance measures for classification tasks. *Information processing & management* 45, 4 (2009), 427–437.

[32] Splunk, 2023. *Define alert suppression groups to throttle sets of similar alerts*. Retrieved Accessed: May 1, 2023 from https://docs.splunk.com/Documentation/Splunk/9.0.4/Alert/SuppressionGroups

[33] Alexandru Todea, Maria Ulici, and Simona Silaghi. 2009. Adaptive markets hypothesis: Evidence from Asia-Pacific financial markets. *The Review of Finance and Banking* 1, 1 (2009).

[34] Alexandru Todea, Adrian Zoicas-Ienciu, and Angela-Maria Filip. 2009. Profitability of the moving average strategy and the episodic dependencies: Empirical evidence from european stock markets. (2009).

[35] R Udhayasankar, N Balaji, Maran Marimuthu, and M Vijay. 2021. A Study On Commodity Market Volatality With Special Reference To Natural Gas. *Ilkogretim Online* 20, 1 (2021), 4790–4803.

[36] Chengwei Wang, Vanish Talwar, Karsten Schwan, and Parthasarathy Ranganathan. 2010. Online detection of utility cloud anomalies using metric distributions. In *2010 IEEE Network Operations and Management Symposium-NOMS 2010*. IEEE, 96–103.

[37] Jingmin Xu, Yuan Wang, Pengfei Chen, and Ping Wang. 2017. Lightweight and adaptive service api performance monitoring in highly dynamic cloud environment. In *2017 IEEE International Conference on Services Computing (SCC)*. IEEE, 35–43.

[38] Nengwen Zhao, Junjie Chen, Xiao Peng, Honglin Wang, Xinya Wu, Yuanzong Zhang, Zikai Chen, Xiangzhong Zheng, Xiaohui Nie, Gang Wang, et al. 2020. Understanding and handling alert storm for online service systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*. 162–171.

[39] Nengwen Zhao, Panshi Jin, Lixin Wang, Xiaoqin Yang, Rong Liu, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2020. Automatically and adaptively identifying severe alerts for online service systems. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*. IEEE, 2420–2429.

[40] Yunyue Zhu and Dennis Shasha. 2003. Efficient elastic burst detection in data streams. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 336–345.